K. J. Somaiya School of Engineering, Mumbai-77

Batch: A2 Roll No.: 16010324044
Experiment / assignment / tutorial No. 7
Grade: AA / AB / BB / BC / CC / CD /DD

Signature of the Staff In-charge with date

TITLE: Virtual Memory Management
Objective: Simulate all page replacement algorithms a)FIFO b) LRU c) OPTIMAL
Outcome: Student can understand different approaches to memory management.

DESCRIPTION: Page replacement algorithms are an important part of virtual memory management and it helps the OS to decide which memory page can be moved out making space for the currently needed page. However, the ultimate objective of all page replacement algorithms is to reduce the number of page faults.

FIFO-This is the simplest page replacement algorithm. In this algorithm, the operating system keeps track of all pages in the memory in a queue, the oldest page is in the front of the queue. When a page needs to be replaced page in the front of the queue is selected for removal.
LRU-In this algorithm page will be replaced which is least recently used
OPTIMAL- In this algorithm, pages are replaced which would not be used for the longest duration of time in the future. This algorithm will give us less page fault when compared to other page replacement algorithms.

ALGORITHM:

1. Start the process
2. Read number of pages n
3. Read number of pages no
4. Read page numbers into an array a[i]
5. Initialize avail[i]=0 .to check page hit
6. Replace the page with circular queue, while re-placing check page availability in the frame

Department of Electronics and Telecommunication Engineering

OS/Sem V/July.-Nov 26 Page No

TITLE: Virtual Memory Management
Objective: Simulate all page replacement algorithms a)FIFO b) LRU c) OPTIMAL
Outcome: Student can understand different approaches to memory management.

DESCRIPTION: Page replacement algorithms are an important part of virtual memory management and it helps the OS to decide which memory page can be moved out making space for the currently needed page. However, the ultimate objective of all page replacement algorithms is to reduce the number of page faults.

FIFO-This is the simplest page replacement algorithm. In this algorithm, the operating system keeps track of all pages in the memory in a queue, the oldest page is in the front of the queue. When a page needs to be replaced page in the front of the queue is selected for removal.
LRU-In this algorithm page will be replaced which is least recently used
OPTIMAL- In this algorithm, pages are replaced which would not be used for the longest duration of time in the future. This algorithm will give us less page fault when compared to other page replacement algorithms.

ALGORITHM:
1. Start the process
2. Read number of pages n
3. Read number of pages no
4. Read page numbers into an array a[i]
5. Initialize avail[i]=0 .to check page hit
6. Replace the page with circular queue, while re-placing check page availability in the frame
Place avail[i]=1 if page is placed in the frame Count page faults

7. Print the results.

8. Stop the process.

Code:

FIFO

```C/C++
#include <stdio.h>


int main() {
    int pages[50], frames[10];
    int n, f;
    int i, j, k = 0;
    int pageFaults = 0, hit;

    printf("Enter number of pages: ");
    scanf("%d", &n);

    printf("Enter page reference string:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &pages[i]);

    printf("Enter number of frames: ");
    scanf("%d", &f);

    for(i = 0; i < f; i++)
        frames[i] = -1;
```

```
for(i = 0; i < n; i++) {
    hit = 0;

    for(j = 0; j < f; j++) {
        if(frames[j] == pages[i]) {
            hit = 1;
            break;
        }
    }

    if(!hit) {
        frames[k] = pages[i];
        k = (k + 1) % f;
        pageFaults++;
    }

    printf("\nPage %d: ", pages[i]);
    for(j = 0; j < f; j++) {
        if(frames[j] == -1)
            printf("- ");
        else
            printf("%d ", frames[j]);
    }
}
```

```
    printf("\n\nTotal Page Faults = %d\n", pageFaults);

    return 0;
}
```

LRU

```
C/C++
#include <stdio.h>

int main() {
    int pages[50], frames[10], recent[10];
    int n, f;
    int i, j, k;
    int pageFaults = 0, hit, pos;

    printf("Enter number of pages: ");
    scanf("%d", &n);

    printf("Enter page reference string:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &pages[i]);

    printf("Enter number of frames: ");
    scanf("%d", &f);
```

```
for(i = 0; i < f; i++) {
    frames[i] = -1;
    recent[i] = 0;
}

for(i = 0; i < n; i++) {
    hit = 0;

    for(j = 0; j < f; j++) {
        if(frames[j] == pages[i]) {
            hit = 1;
            recent[j] = i;
            break;
        }
    }

    if(!hit) {
        pageFaults++;

        pos = 0;

        for(j = 1; j < f; j++) {
            if(recent[j] < recent[pos])
```

```
                    pos = j;
                }

                frames[pos] = pages[i];
                recent[pos] = i;
            }

            printf("\nPage %d: ", pages[i]);
            for(k = 0; k < f; k++) {
                if(frames[k] == -1)
                    printf("- ");
                else
                    printf("%d ", frames[k]);
            }
        }

        printf("\n\nTotal Page Faults = %d\n", pageFaults);

        return 0;
}
```

Optimal

```
C/C++
#include <stdio.h>
```

```c
int main() {
    int pages[50], frames[10];
    int n, f;
    int i, j, k;
    int pageFaults = 0, hit;
    int pos, farthest, next;

    printf("Enter number of pages: ");
    scanf("%d", &n);

    printf("Enter page reference string:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &pages[i]);

    printf("Enter number of frames: ");
    scanf("%d", &f);

    for(i = 0; i < f; i++)
        frames[i] = -1;

    for(i = 0; i < n; i++) {
        hit = 0;

        for(j = 0; j < f; j++) {
```

```
        if(frames[j] == pages[i]) {
            hit = 1;
            break;
        }
    }

    if(!hit) {
        pageFaults++;

        /* Empty frame available */
        pos = -1;

        for(j = 0; j < f; j++) {
            if(frames[j] == -1) {
                pos = j;
                break;
            }
        }

        /* Find page used farthest in future */
        if(pos == -1) {
            farthest = -1;

            for(j = 0; j < f; j++) {
```

```
                next = n + 1;

                for(k = i + 1; k < n; k++) {
                    if(frames[j] == pages[k]) {
                        next = k;
                        break;
                    }
                }

                if(next > farthest) {
                    farthest = next;
                    pos = j;
                }
            }
        }

        frames[pos] = pages[i];
    }

    printf("\nPage %d: ", pages[i]);

    for(j = 0; j < f; j++) {
        if(frames[j] == -1)
            printf("- ");
```

```
            else
                printf("%d ", frames[j]);
        }
    }

    printf("\n\nTotal Page Faults = %d\n", pageFaults);

    return 0;
}
```

Output:

FIFO

```
Enter number of pages: 7
Enter page reference string:
1 2 3 4 1 2 5
Enter number of frames: 3

Page 1: 1 - -
Page 2: 1 2 -
Page 3: 1 2 3
Page 4: 4 2 3
Page 1: 4 1 3
Page 2: 4 1 2
Page 5: 5 1 2

Total Page Faults = 7
```

LRU

```
Enter number of pages: 5
Enter page reference string:
1 2 3 1 4
Enter number of frames: 3

Page 1: 1 - -
Page 2: 2 - -
Page 3: 2 3 -
Page 1: 2 3 1
Page 4: 4 3 1

Total Page Faults = 5
```

Optimal

```
Enter number of pages: 6
Enter page reference string:
1 4 5 1 2 3
Enter number of frames: 3

Page 1: 1 - -
Page 4: 1 4 -
Page 5: 1 4 5
Page 1: 1 4 5
Page 2: 2 4 5
Page 3: 3 4 5

Total Page Faults = 5
```

Conclusion:

In this experiment, I successfully implemented FIFO, LRU and Optimal page replacement algorithms. The algorithms determine which page should be replaced when a page fault occurs. FIFO replaces the oldest page, LRU replaces the least recently used page, and Optimal replaces the page that will be used farthest in the future. The number of page faults was calculated for each algorithm and their performance can be compared based on the page-fault count.